

Exercise 7: An Agent Team Builds Your Spec

Session: 7 — Agentic Workflows **Duration:** 30 minutes **Project:** `bookstore-web` — the frontend you specified in Exercise 6.

Goal

In Exercise 6 you produced a spec, a plan and a task list, and no code. This session an agent team builds it. You build the shared foundation in one session, then draft a coached prompt that hands the user stories to a team, one teammate per story. While they work you watch them. Then you check the result against the constitution and the spec.

You will not write application code yourself.

A few words we'll use

- **Foundation:** Phase 1 and Phase 2 of `tasks.md`. The files every story needs: the API client, the page shell, the stylesheet, error rendering. The constitution says stories may not start before it is done.
- **Lead:** the Claude Code session you type in. It spawns teammates, hands out tasks and collects results.
- **Teammate:** a separate Claude Code session started by the lead. It has its own context window and its own task. You can open its transcript and talk to it directly.
- **Agent team:** the lead plus its teammates, sharing one task list.
- **Coach:** the `/parallel-coach` command. It grades your draft prompt and points out gaps. It never writes the prompt for you — that's your job.
- **Greenlight:** the coach thinks your prompt is ready to run.
- **Verifier:** the `/verify-exercise 7` command. It checks what the team built against the constitution and your spec, after the fact.

Agent teams are experimental. They are off unless a flag is set. This project sets the flag in `bookstore-web/.claude/settings.json`, which you checked during preparation. If teammates never appear in task 3, tell the trainer — do not spend your time on it.

Before you start

Do you have a spec? Exercise 7 needs the `specs/001-*/` folder you committed at the end of Exercise 6. Check:

```
cd ../bookstore-web && ls specs/
```

No folder, or an unfinished one? Use the reference spec instead. It was written for this exercise and follows the same format:

```
cp -r specs-reference/001-browse-books specs/
git add specs/ && git commit -m "spec: reference spec for exercise 7"
```

You need three terminals, the same as Exercise 6.

Terminal 1 — your backend, from the `bookstore-go` project directory:

```
go run .
```

Terminal 2 — Claude Code, started from `bookstore-web` :

```
cd ../bookstore-web && claude
```

Terminal 3 — a plain shell, for `npm run dev` , `curl` and `git` .

Keep a browser tab ready for <http://localhost:5173>. It shows an empty page until task 1 is done.

Tasks

1. Build the foundation, one session (6 min)

The constitution says shared code is built before any story starts. That part is not parallel work. One session does it.

Open `specs/001-*/tasks.md` and find **Phase 1** and **Phase 2**. Write down, in terminal 3 or on paper, the list of files those phases create. Keep the list — task 2 needs it, and so does the verifier.

Then send the lead this prompt. Copy it as written:

```
Implement Phase 1 and Phase 2 of specs/001-*/tasks.md , and nothing from any user story. Follow
.specify/memory/constitution.md . Tick each task off in tasks.md when it is done. When you
finish, list the files you created.
```

It runs for two to four minutes. **Do not wait. Start task 2 now** — drafting a prompt does not touch the tree, so it cannot collide with this run.

When the lead reports back, in terminal 3:

```
cd ../bookstore-web && npm run dev
```

Open <http://localhost:5173>. You should see the page shell, and no errors in the browser console. Then commit, so the verifier can see where the foundation ends and story work begins:

```
git add -A && git commit -m "foundation" && git tag foundation
```

Done when: Phase 1 and 2 are ticked in `tasks.md` , the page shell loads on port 5173, and the foundation is committed.

2. Draft the team prompt (9 min: draft 4 · coach & revise 5)

This is the core of the exercise. You write one prompt that turns the lead into a team lead. `tasks.md` already contains the staffing plan — find **Parallel Team Strategy** near the bottom. Your prompt has to turn that plan into instructions a lead can follow.

Think about what a teammate does not know. It loads the constitution and the project files, but nothing from your chat. It has no idea which files are "finished" unless the prompt says so. And a lead that gets bored waits badly: it may start building a story itself.

A worked example from another project, to steal the shape from — not the content:

Read `specs/002-*/tasks.md` . Spawn an agent team with one teammate per user story, named after the story: `us1-search` , `us2-export` . Each teammate claims only the tasks tagged with its story. The files from Phase 1 and 2 (`src/api.js` , `src/main.js` , `styles.css`) are finished: nobody edits them. Each teammate is done when its story's Independent Test in `tasks.md` passes against the running backend and its tasks are ticked. You wait for both teammates. You do not implement any story yourself. Report per story: files changed, and the result of its Independent Test.

Draft your own version for your `tasks.md` . Then:

Draft → `/parallel-coach 2` → revise until you get the greenlight.

Disagree with the coach? Say "run it anyway" — it will let you, and tell you what to watch for.

Done when: the coach greenlit your team prompt, and task 1 is done.

3. Launch the team and watch it (8 min)

Paste your greenlit prompt into the lead, in terminal 2. The coach does not run this one for you — a team can only start from the session you are typing in.

Within a minute, the **agent panel** appears below the prompt input, with one row per teammate. If nothing appears after a minute, Claude used plain subagents instead. Say: "Use an agent team, not subagents."

Now watch, and do three things:

1. Press **Ctrl+T**. That is the shared task list. Which tasks are claimed, and by whom?
2. Use the **up and down arrows** to select a teammate, then **Enter**. You are now in its transcript. Read what it is doing. To leave, use the arrows to select the lead's row again. Do not press **Escape** inside a transcript: that interrupts the teammate. You can type to it here — do not, unless it is stuck.
3. Refresh <http://localhost:5173> every minute. Stories appear while you watch.

Watch the lead too. If it starts editing story files itself, tell it: "Wait for your teammates to finish."

Done when: both teammates report done, every story task in `tasks.md` is ticked, and both stories show in the browser.

4. Quality gates (7 min: check 4 · verify 3)

Nobody has reviewed this code. Three checks, in order of what a reviewer would find first.

Does it do what the spec says? Open `specs/001-*/quickstart.md` and walk its steps in the browser. Then the three gaps from Exercise 6, task 2:

- Reach page 2 of the book list. Does it show different books?
- Go past the last page. What does an empty page show?
- Open a book id that does not exist, such as `#/books/9999` . What do you see — and is it the raw error text from the backend?

Did the team respect the file boundaries? In terminal 3:

```
git diff --stat foundation
```

Every changed file should belong to one story. A foundation file in that list means a teammate edited code it did not own.

Is it still contract-only? One grep:

```
grep -rn "8080\|localhost" src/ index.html
```

Any hit is a constitution violation, principle II.

Write down what you found. Then start `/verify-exercise 7` and paste your team prompt when it asks. It grades the prompt first, then checks the tree for the same three things, plus the error paths from principle V. Bring its report to the closing round.

Commit:

```
git add -A && git commit -m "feat: bookstore-web, built by agent team"
```

Done when: you walked the quickstart, ran the two commands, and `/verify-exercise 7` has run.

Closing round (5 min)

Trainer popcorns the room — have answers ready:

- What did the coach flag in your team prompt before it greenlit it?
 - Did the lead wait, or did it start building a story itself?
 - Did a teammate touch a file it did not own? Which one, and why?
 - In Exercise 6 you marked some answers as guesses. Find one in the running frontend. Does the code treat it as a fact?
 - Close with **one take-away**: which part of this exercise needed a team, and which part would have been faster in a single session?
-

What you should have

bookstore-web/	
index.html, styles.css, src/	built from your spec
specs/001-*/tasks.md	every task ticked
git log	"spec" → "foundation" → "feat" commits

Every line of application code in this project was written from a spec you wrote, by a session you never typed in.